

Create a complete end-to-end test suite for my SaaS web app "Zakerly".

Project overview:

Zakerly is an Arabic RTL SaaS platform for teachers to create online homework and share it with students using a link.

Teachers sign up, log in, create homework, publish it, copy the share link, and view student results.

Students open the public link, enter their name, solve the homework, submit answers, and instantly see their score and ranking.

Testing goals:

Test the full SaaS flow end-to-end across authentication, dashboard, homework creation, publishing, public student solving flow, results tracking, permissions, and edge cases.

Important requirements:

- The app is in Arabic and uses RTL layout
- Test both desktop and mobile responsive behavior where relevant
- Prefer stable selectors and robust assertions
- Cover success cases, failure cases, validation, and permissions
- Keep tests readable, modular, and maintainable
- If test data is needed, create it dynamically during the tests
- Clean up or isolate test data where possible

Test these flows:

1. Landing Page

- Verify the landing page loads correctly

- Verify Arabic text is visible
- Verify main CTA button exists
- Verify navigation and basic layout render properly

2. Teacher Sign Up

- Create a new teacher account with unique test email
- Verify successful signup flow
- Verify redirect behavior after signup
- Verify profile creation works correctly

3. Teacher Login

- Log in with valid credentials
- Verify dashboard access
- Test invalid login credentials
- Verify proper error messages appear

4. Protected Routes

- Verify unauthenticated users cannot access teacher dashboard pages
- Verify protected routes redirect correctly
- Verify authenticated session persists correctly

5. Teacher Dashboard

- Verify dashboard loads real homework data
- Verify empty state appears correctly when there are no homeworks
- Verify homework cards render correctly after creating homework

6. Create Homework

- Create a homework with title and multiple-choice questions
- Verify form validation
- Verify questions are saved correctly
- Verify publish flow works
- Verify unique share link is generated

7. Publish Homework

- Verify published homework shows success state
- Verify copy link action works if implemented
- Verify homework becomes accessible via public share link
- Verify unpublished homework is not publicly accessible

8. Student Start Page

- Open homework via share link in a fresh unauthenticated session
- Verify homework title appears
- Verify student can enter name
- Verify validation for empty student name

9. Student Homework Flow

- Solve homework as a student
- Submit answers
- Verify result page shows score
- Verify ranking is displayed
- Verify one-attempt rule works if implemented
- Verify duplicate submission is blocked with correct error handling

10. Teacher Results Page

- Return to teacher dashboard
- Open results page for the same homework
- Verify student submission appears
- Verify score is correct
- Verify ranking/order is correct
- Verify solved vs unsolved stats are accurate if shown

11. Authorization / Security

- Verify one teacher cannot access another teacher's homework/results
- Verify public users cannot access teacher-only pages
- Verify invalid or fake share codes show proper not found or unavailable state

12. Validation & Edge Cases

- Homework with missing title
- Question with missing correct answer
- Submitting student form without name
- Invalid share link
- Already submitted student retry
- Homework with zero questions if the UI allows it
- Network/error state handling if present

13. Arabic UI / UX Checks

- Verify major screens render Arabic text properly
- Verify RTL layout is applied on main pages

- Verify important buttons and labels are visible and usable

14. Mobile Responsiveness

- Test key student flow on mobile viewport
- Test key teacher flow on tablet/mobile viewport where relevant
- Verify main interactions remain usable

Suggested end-to-end scenarios:

- Full happy path: teacher signup → login → create homework → publish → student solves → teacher sees result
- Existing teacher login → create second homework → publish → multiple students solve → verify ranking
- Unauthorized user tries to access protected routes
- Student opens invalid share link
- Student tries duplicate submission

Test quality expectations:

- Use clear assertions
- Use reusable helpers where possible
- Use stable selectors
- Prefer behavior-based checks over brittle UI-only checks
- Add comments for major flows
- Keep tests organized by feature area

Deliverables:

- A full end-to-end test suite covering the major SaaS flows
- Clear test grouping by feature

- Good coverage of success, validation, edge, and authorization cases
- Tests ready to run against the current app